

Pythonia

Manual de estudio: tronco común

Constructor

Nivel 2: Intermedio

Código organizado, reutilizable y que no se rompe.

2

10 lecciones | 8 semanas | 1 proyecto integrador

Repasa, practica y registra cada ejecución en tu Cuadernillo-Diario.

Antes de empezar

Dominar las herramientas que separan un script de un programa: funciones flexibles, archivos, excepciones, programación orientada a objetos, entornos y pruebas.

Cada lección tiene la misma estructura: explicación breve, un ejemplo para escribir y ejecutar, la salida real que produce, consejos, un ejercicio y una autoevaluación (respuestas al final). Desde este nivel, cada lección tiene además un **ejercicio con lente** de tu ruta de especialización, reunidos en el anexo final.

Además, cada lección tiene tres apartados de profundización (**el concepto a fondo**, **el ejemplo línea por línea** y **errores frecuentes**) y una **práctica evaluable**: un ejercicio que escribes en la app y se corrige automáticamente con pruebas. Cada práctica es una pieza del proyecto del nivel, que se va armando solo a medida que avanzas.

Método: lee en la app, escribe el ejemplo a mano, predice la salida, ejecútalo y anota el resultado en una ficha de **Registro de ejecución** del Cuadernillo-Diario. Cuando apruebes la práctica, regístrala en la **Bitácora de prácticas**: intentos, pistas usadas, el error que más te costó y qué aprendiste.

Tu plan

Semana	Temas	Misión de la semana
1	1. Comprensiones 2. Funciones avanzadas	Escribe una función <code>crear_etiqueta(texto, **estilo)</code> que devuelva el texto rodeado por los símbolos y el ancho que se pasen por kwargs.
2	3. Módulos y paquetes 4. Archivos, CSV y JSON	Lee un CSV de ventas (producto, cantidad, precio) y genera un JSON con el total vendido por producto.
3	5. Excepciones a fondo 6. Programación orientada a objetos I	Modela una <code>CuentaBancaria</code> con titular y saldo, y métodos <code>depositar</code> , <code>retirar</code> y <code>mostrar_saldo</code> .
4	7. Programación orientada a objetos II	Crea <code>Empleado</code> y dos subclases (<code>Vendedor</code> con comisión, <code>Gerente</code> con bono) que calculen su sueldo de forma distinta.
5	8. Entornos virtuales y pip	Crea un entorno para tu proyecto, instala 'rich' y úsalo para imprimir una tabla con colores.
6	9. Pruebas con assert y pytest	Escribe 5 tests para tu función <code>es_primo</code> , incluyendo 0, 1, 2 y un número grande.
7	10. Depuración profesional	Toma un programa tuyo, introduce un bug a propósito y encuéntralo usando <code>breakpoint()</code> .
8	Proyecto integrador: Gestor de tareas de consola	Construir, probar y documentar el proyecto

Ritmo sugerido: 30 a 45 minutos al día, 5 días por semana. Lo importante es no cortar la racha.

1 Comprensiones

Una comprensión crea una colección en una sola línea: `[expresión for x in iterable if condición]`.

Existen para listas `[]`, diccionarios `{k: v}` y conjuntos `{}`. Son más rápidas y legibles que un `for` con `append`, siempre que no se vuelvan demasiado largas.

Si una comprensión no entra cómoda en una línea o tiene más de un `if`, vuelve al `for` clásico.

El concepto a fondo

Una comprensión de lista construye una lista nueva a partir de otra en una sola expresión: `[expresión for elemento in iterable if condición]`. Se lee casi en castellano: "el cuadrado de `n`, para cada `n` del rango, si `n` es par".

Equivale a un bucle `for` con `append`, pero es más corta, más rápida y deja claro que el objetivo es crear una colección. La condición `if` es opcional y filtra; la expresión del principio transforma.

Existen también comprensiones de diccionario (`{clave: valor for ...}`) y de conjunto (`{valor for ...}`). Con paréntesis en lugar de corchetes obtienes una expresión generadora, que produce los valores de `a` uno sin crear la lista entera (la verás a fondo en el Nivel 3).

Regla de estilo: si la comprensión no entra cómoda en una línea o necesita varios `if` anidados, vuelve al `for` clásico. La claridad siempre gana.

Ejemplo

```
1 cuadrados = [n**2 for n in range(6) if n % 2 == 0]
2 print(cuadrados)
3
4 invertido = {v: k for k, v in {"a": 1, "b": 2}.items()}
5 print(invertido)
6
7 iniciales = {nombre[0] for nombre in ["Ana", "Alan", "Bea"]}
8 print(sorted(iniciales))
```

Salida real

```
[0, 4, 16]
{1: 'a', 2: 'b'}
['A', 'B']
```

El ejemplo, línea por línea

1. `[n**2 for n in range(6) if n % 2 == 0]` toma 0, 2 y 4 y los eleva al cuadrado.
2. La comprensión de diccionario invierte claves y valores recorriendo `.items()`.
3. La comprensión de conjunto junta las iniciales sin repetir; `sorted` las ordena para imprimirlas.

Consejos

- Legibilidad primero: una comprensión clara vale más que una 'ingeniosa'.
- Con paréntesis obtienes un generador: `sum(x*x for x in datos)` no crea una lista en memoria.

Ejercicio

A partir de una lista de precios, crea con comprensiones: precios con IVA, solo los mayores a 100 y un dict producto→precio.

Errores frecuentes

Error frecuente	Cómo resolverlo
SyntaxError en una comprensión con else	En una comprensión el if filtrante va al final. Si quieres elegir entre dos valores usa <code>x if cond else y</code> al principio: <code>["par" if n % 2 == 0 else "impar" for n in nums]</code> .
Comprensión ilegible	Si no entiendes tu comprensión al releerla, conviértela en un <code>for</code> normal.

Práctica evaluable**Filtrar tareas** · Pieza del proyecto: **Filtros con comprensiones**

En el Gestor de tareas, cada tarea es un diccionario como `{"titulo": "Comprar pan", "prioridad": "alta", "hecha": False}`. Escribe con comprensiones:

- `pendientes(tareas)`: lista con los títulos de las tareas no hechas.
- `por_prioridad(tareas, prioridad)`: lista con las tareas (diccionarios) de esa prioridad.
- `indice(tareas)`: diccionario `{titulo: tarea}`.
- `prioridades_usadas(tareas)`: conjunto con las prioridades que aparecen.

Plantilla para empezar

```

1 def pendientes(tareas):
2     """Títulos de las tareas que no están hechas."""
3     return [t["titulo"] for t in tareas if not t["hecha"]]
4
5
6 def por_prioridad(tareas, prioridad):
7     """Tareas (diccionarios completos) con esa prioridad."""
8
9
10 def indice(tareas):
11     """Diccionario {titulo: tarea}."""
12
13
14 def prioridades_usadas(tareas):
15     """Conjunto de prioridades presentes."""

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

`[c for c in "hola" if c != "o"]` da...

- `['h','o','l','a']`
- `'hla'`
- `['o']`
- `['h','l','a']`

2 Funciones avanzadas

`*args` recoge argumentos posicionales extra en una tupla; `**kwargs` recoge argumentos con nombre en un diccionario. Así creas funciones flexibles.

`lambda` crea funciones pequeñas y anónimas, ideales como parámetro `key` de `sorted`, `max` o `min`.

Ámbito (scope): Python busca nombres en orden Local → Envolverte → Global → Built-in (regla LEGB).

El concepto a fondo

`*args` reúne todos los argumentos posicionales extra en una tupla, y `**kwargs` todos los argumentos con nombre extra en un diccionario. Los nombres `args` y `kwargs` son convención; lo que importa son los asteriscos.

El orden de los parámetros es fijo: primero los normales, después `*args`, después los que tienen nombre (como `prioridad="media"`, que tras `*args` solo pueden pasarse por nombre) y al final `**kwargs`.

Una `lambda` es una función anónima de una sola expresión: `lambda s: len(s)`. Brillan como argumento `key` de `sorted`, `min` y `max`, donde indican por qué criterio comparar.

Los asteriscos también sirven al llamar: `f(*lista)` reparte una lista como argumentos posicionales y `f(**diccionario)` reparte un diccionario como argumentos con nombre. Y `{**a, **b}` une diccionarios.

Ejemplo

```
1 def resumen(*nums, **opciones):
2     total = sum(nums)
3     if opciones.get("redondear"):
4         total = round(total)
5     return total
6
7 print(resumen(1, 2, 3.5))
8 print(resumen(1, 2, 3.5, redondear=True))
9
10 orden = sorted(["kiwi", "uva", "banana"], key=lambda s: len(s))
11 print(orden)
```

Salida real

```
6.5
6
['uva', 'kiwi', 'banana']
```

El ejemplo, línea por línea

1. `*nums` recibe 1, 2 y 3.5 como una tupla; `**opciones` recibe `redondear=True` como diccionario.
2. `opciones.get("redondear")` devuelve `None` si no se pasó, así que no redondea.
3. `sorted(..., key=lambda s: len(s))` ordena las palabras por su longitud.

Consejos

- ¿Viste que `round(6.5)` da 6? Python usa redondeo bancario: los .5 van al par más cercano.
- Para dinero usa el módulo `decimal`, no `float`.

Ejercicio

Escribe una función `crear_etiqueta(texto, **estilo)` que devuelva el texto rodeado por los símbolos y el ancho que se pasen por kwargs.

Errores frecuentes

Error frecuente	Cómo resolverlo
<code>TypeError: got multiple values for argument</code>	Pasaste el mismo parámetro por posición y por nombre a la vez.
Una lambda con varias líneas	Las lambdas admiten una sola expresión. Para algo más largo, define una función con <code>def</code> .

Práctica evaluable**Crear tareas flexibles** · Pieza del proyecto: **Creación y orden de tareas**

- `crear_tarea(titulo, *etiquetas, prioridad="media", **extra)`: devuelve el diccionario de la tarea con el título sin espacios en los bordes, la prioridad, "hecha": False, las etiquetas como lista y además cualquier dato extra que se pase por nombre.

```
crear_tarea("Pagar luz", "casa", prioridad="alta", responsable="Ana") →
{"titulo": "Pagar luz", "prioridad": "alta", "hecha": False, "etiquetas": ["casa"],
"responsable": "Ana"}
```

- `ordenar(tareas, por="titulo", descendente=False)`: devuelve una lista nueva ordenada por la clave indicada, usando `sorted` con una lambda.

Plantilla para empezar

```
1 def crear_tarea(titulo, *etiquetas, prioridad="media", **extra):
2     """Crea el diccionario de una tarea."""
3     tarea = {"titulo": titulo.strip(), "prioridad": prioridad, "hecha": False}
4     # agrega las etiquetas como lista y los datos extra
5     return tarea
6
7
8 def ordenar(tareas, por="titulo", descendente=False):
9     """Lista nueva ordenada por la clave `por`."""
```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

`(lambda x, y: x * y)(3, 4)` devuelve...

- a) 7
- b) Error
- c) 12
- d) 34

3 Módulos y paquetes

Cada archivo `.py` es un módulo. Con `import` reutilizas código propio o de la biblioteca estándar (`math`, `random`, `datetime`, `pathlib`, `json`, `csv`...).

Un paquete es una carpeta de módulos. Organizar tu proyecto en módulos mantiene cada archivo corto y enfocado.

El bloque `if __name__ == "__main__":` ejecuta código solo cuando el archivo se corre directamente, no cuando se importa.

El concepto a fondo

Cada archivo `.py` es un módulo, y una carpeta con módulos es un paquete. `import math` trae el módulo completo (lo usas como `math.sqrt`); `from random import randint` trae solo un nombre (lo usas directo).

La biblioteca estándar viene incluida con Python y resuelve muchísimo: `math`, `random`, `datetime`, `collections`, `json`, `pathlib`, `re`... Antes de programar algo genérico, busca si ya existe.

`collections.Counter` cuenta elementos en una línea: `Counter(["a", "b", "a"])` da `{"a": 2, "b": 1}`. Se comporta como un diccionario y además tiene `most_common()`.

Al dividir tu proyecto en varios archivos, el bloque `if __name__ == "__main__":` hace que cierto código se ejecute solo cuando corres ese archivo directamente, y no cuando otro archivo lo importa.

Ejemplo

```
1 import math
2 from random import seed, randint
3 from datetime import date
4
5 print(math.sqrt(16), round(math.pi, 2))
6 seed(42)
7 print(randint(1, 10))
8 print(date(2026, 1, 1).strftime("%d/%m/%Y"))
```

Salida real

```
4.0 3.14
2
01/01/2026
```

El ejemplo, línea por línea

1. `import math` y luego `math.sqrt(16)` da 4.0.
2. `seed(42)` fija la semilla: `randint` dará siempre el mismo número, útil para pruebas reproducibles.
3. `strftime("%d/%m/%Y")` da formato día/mes/año a una fecha.

Consejos

- Evita `from modulo import *`: no se sabe de dónde viene cada nombre.
- `seed()` hace que lo aleatorio sea reproducible, útil para pruebas.

Ejercicio

Separa tu juego 'Adivina el número' en dos módulos: logica.py (funciones) y main.py (interacción).

Errores frecuentes

Error frecuente	Cómo resolverlo
ModuleNotFoundError: No module named 'requests'	El módulo no está instalado en tu entorno: instálalo con pip (lección 2.8).
Tu archivo se llama random.py	Si nombras un archivo igual que un módulo estándar, lo tapas. Cámbiale el nombre.

Práctica evaluable**Resumen con la biblioteca estándar · Pieza del proyecto: Resumen y estadísticas**

Usa módulos de la biblioteca estándar:

- `resumen(tareas)`: devuelve `{"total": 4, "hechas": 1, "pendientes": 3, "por_prioridad": {"alta": 2, "media": 1, "baja": 1}}`. Cuenta las prioridades con `collections.Counter` y devuelve `por_prioridad` como diccionario común.
- `porcentaje_hecho(tareas)`: porcentaje entero de tareas hechas, redondeado hacia abajo con `math.floor`. Con una lista vacía devuelve 0.
- `fecha_corta(fecha)`: recibe un objeto `date` y devuelve el texto `"02/10/2026"`.

Plantilla para empezar

```

1  from collections import Counter
2  import math
3
4
5  def resumen(tareas):
6      """Totales y conteo por prioridad."""
7      hechas = sum(1 for t in tareas if t["hecha"])
8
9
10 def porcentaje_hecho(tareas):
11     """Porcentaje entero (hacia abajo); 0 si no hay tareas."""
12
13
14 def fecha_corta(fecha):
15     """date -> 'dd/mm/aaaa'."""

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

Al ejecutar un archivo directamente, `__name__` vale...

- el nombre del archivo
- "main"
- None
- "__main__"

4 Archivos, CSV y JSON

`pathlib.Path` es la forma moderna de manejar rutas: funciona igual en Windows, Linux y macOS.

`with open(...)` as `f`: abre un archivo y lo cierra automáticamente, aun si ocurre un error. Indica siempre `encoding="utf-8"`.

`json` convierte entre diccionarios y texto JSON; `csv` lee y escribe tablas. Son la base de casi cualquier automatización.

El concepto a fondo

`pathlib.Path` representa una ruta y funciona igual en Windows, macOS y Linux. Se combinan rutas con `/` (carpeta / "datos.json") y se lee o escribe con `read_text` y `write_text`, indicando siempre `encoding="utf-8"` para que tildes y eñes se guarden bien.

`with open(ruta, encoding="utf-8")` as `f`: es la forma clásica de abrir archivos: el `with` garantiza que el archivo se cierre aunque ocurra un error en el medio.

JSON es el formato de intercambio más usado: guarda diccionarios, listas, textos, números, `true`, `false` y `null`. `json.dumps` convierte un objeto de Python en texto JSON y `json.loads` hace lo contrario. Con `ensure_ascii=False` las eñes se ven tal cual, y con `indent=2` el archivo queda legible.

CSV guarda tablas en texto plano, una fila por línea. El módulo `csv` (con `DictReader` y `DictWriter`) maneja comillas y separadores por ti. En el Nivel 3 de las rutas de datos lo harás con `pandas`.

Ejemplo

```
1 from pathlib import Path
2 import json
3
4 ruta = Path("notas.json")
5 datos = {"alumno": "Leo", "notas": [8, 9, 7]}
6 ruta.write_text(json.dumps(datos, ensure_ascii=False), encoding="utf-8")
7
8 cargado = json.loads(ruta.read_text(encoding="utf-8"))
9 print(cargado["alumno"], sum(cargado["notas"]) / len(cargado["notas"]))
10 print(ruta.exists(), ruta.suffix)
```

Salida real

```
Leo 8.0
True .json
```

El ejemplo, línea por línea

1. `Path("notas.json")` crea la ruta.
2. `json.dumps(datos, ensure_ascii=False)` convierte el diccionario en texto y `write_text` lo guarda.
3. `json.loads(...read_text())` lo vuelve a convertir en diccionario.
4. `ruta.exists()` y `ruta.suffix` consultan la ruta.

Consejos

- Para CSV usa `csv.DictReader`: cada fila llega como diccionario con los encabezados como claves.
- `ensure_ascii=False` conserva tildes y ñes legibles en el JSON.

Ejercicio

Lee un CSV de ventas (producto, cantidad, precio) y genera un JSON con el total vendido por producto.

Errores frecuentes

Error frecuente	Cómo resolverlo
UnicodeDecodeError al leer	Falta <code>encoding="utf-8"</code> o el archivo usa otra codificación.
<code>json.decoder.JSONDecodeError</code>	El texto no es JSON válido: suele faltar una coma o hay comillas simples en lugar de dobles.

Práctica evaluable

Guardar y cargar en JSON · Pieza del proyecto: **Persistencia en JSON**

El programa principal guarda las tareas en `tareas.json` usando tus funciones:

- `a_json(tareas)`: devuelve el texto JSON con `indent=2` y conservando tildes y ñes.
- `desde_json(texto)`: devuelve la lista de tareas. Si el texto está vacío o solo tiene espacios, devuelve `[]` (archivo nuevo). Si el JSON no contiene una lista, lanza `ValueError("El archivo no contiene una lista de tareas")`.

Plantilla para empezar

```

1 import json
2
3
4 def a_json(tareas):
5     """Lista de tareas -> texto JSON legible."""
6
7
8 def desde_json(texto):
9     """Texto JSON -> lista de tareas ([]) si está vacío."""

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Por qué usar `with open(...)`?

- Cierra el archivo automáticamente, incluso con errores
- Es obligatorio
- Permite leer binarios
- Es más rápido

5 Excepciones a fondo

La estructura completa es `try / except / else / finally`. `else` corre solo si no hubo error; `finally` corre siempre (ideal para liberar recursos).

Con `raise` lanzas tus propios errores cuando algo no tiene sentido para tu programa.

Crear excepciones propias (heredando de `Exception`) hace que los errores de tu dominio sean claros y fáciles de atrapar.

El concepto a fondo

La estructura completa es `try / except / else / finally`. `except` atrapa el error; `else` corre solo si no hubo error; `finally` corre siempre, haya o no error, y es ideal para cerrar recursos o registrar que la operación terminó.

Puedes atrapar varios tipos a la vez con una tupla, `except (ValueError, KeyError):`, y guardar el error en una variable con `as e` para mostrar su mensaje.

`raise` lanza un error a propósito. Crear tus propias excepciones (`class TareaInvalida(Exception): pass`) hace el código expresivo: quien llama puede atrapar exactamente ese problema sin confundirlo con otros.

Principio clave: valida en el borde (donde entran los datos) y deja que los errores inesperados se vean. Atrapar todo para "que no se caiga" esconde fallas que después cuestan mucho más encontrar.

Ejemplo

```

1 class SaldoInsuficiente(Exception):
2     pass
3
4 def retirar(saldo, monto):
5     if monto > saldo:
6         raise SaldoInsuficiente(f"Faltan {monto - saldo}")
7     return saldo - monto
8
9 try:
10     retirar(100, 150)
11 except SaldoInsuficiente as e:
12     print("Error:", e)
13 else:
14     print("Retiro OK")
15 finally:
16     print("Operación registrada")

```

Salida real

```

Error: Faltan 50
Operación registrada

```

El ejemplo, línea por línea

1. `SaldoInsuficiente` hereda de `Exception`: es un error propio.
2. `retirar` lanza el error con un mensaje calculado.
3. El `except` lo atrapa; `else` no corre porque hubo error; `finally` corre igual.

Consejos

- Mensajes de error claros: qué pasó, con qué dato y, si se puede, cómo arreglarlo.
- `raise ... from` encadena errores y conserva la causa original.

Ejercicio

Crea una función `transferir()` que lance excepciones propias para monto negativo y saldo insuficiente, y pruébalas.

Errores frecuentes

Error frecuente	Cómo resolverlo
Tu excepción no se atrapa	Revisa que <code>except</code> nombre la misma clase que se lanza (o una clase madre).
Se pierde el mensaje del error	Usa <code>except TipoError as e:</code> y muestra <code>e</code> , o vuelve a lanzar con <code>raise</code> .

Práctica evaluable

Validar tareas con excepciones propias · Pieza del proyecto: **Validación con excepciones**

- Crea la excepción `TareaInvalida`.
- `validar(tarea)`: lanza `TareaInvalida` con el mensaje "La tarea necesita un título" si el título está vacío o solo tiene espacios, o con "Prioridad inválida: urgente" (con la prioridad real) si no es alta, media o baja. Si todo está bien, devuelve `True`.
- `agregar_segura(tareas, tarea, errores)`: intenta validar. Si es válida la agrega a `tareas` y devuelve `True`; si no, agrega el mensaje del error a la lista `errores` y devuelve `False`. Usa `try / except / else`.

Plantilla para empezar

```

1 PRIORIDADES = ("alta", "media", "baja")
2
3
4 class TareaInvalida(Exception):
5     """Error propio del gestor de tareas."""
6
7
8 def validar(tarea):
9     if not tarea["titulo"].strip():
10         raise TareaInvalida("La tarea necesita un título")
11     # revisa la prioridad
12     return True
13
14
15 def agregar_segura(tareas, tarea, errores):
16     """Agrega si es válida; si no, anota el error. Devuelve True o False."""

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Cuándo se ejecuta el bloque else de un try?

- a) Solo si hubo excepción
- b) Siempre
- c) Nunca
- d) Solo si no hubo excepción

6**Programación orientada a objetos I**

Una clase es un molde; un objeto (instancia) es algo creado con ese molde. La clase define atributos (datos) y métodos (comportamiento).

`__init__` inicializa cada objeto nuevo. `self` es la referencia al objeto actual y debe ser el primer parámetro de los métodos.

Los atributos de clase se comparten entre todas las instancias; los de instancia (`self.x`) son propios de cada objeto.

El concepto a fondo

Una clase es un molde que agrupa datos (atributos) y comportamiento (métodos). Cada objeto creado con ese molde es una instancia con sus propios valores: dos tareas comparten la forma, no el contenido.

`__init__` es el inicializador: corre automáticamente al crear el objeto y prepara sus atributos. `self` es la instancia misma; por eso los atributos se guardan como `self.titulo`, y todo método recibe `self` como primer parámetro.

Los atributos de instancia (`self.energia`) son propios de cada objeto; los atributos de clase (definidos directamente en la clase) se comparten entre todas las instancias.

Los métodos especiales, con doble guion bajo, conectan tu clase con Python: `__str__` define cómo se ve al imprimirla, `__repr__` cómo se ve al depurar y `__eq__` cómo se compara con `==`. Los verás a fondo en el Nivel 4.

Ejemplo

```

1 class Mascota:
2     especie = "desconocida" # atributo de clase
3
4     def __init__(self, nombre, energia=5):
5         self.nombre = nombre
6         self.energia = energia
7
8     def jugar(self):
9         self.energia -= 2
10        return f"{self.nombre} juega. Energía: {self.energia}"
11
12 p = Mascota("Pyto")
13 print(p.jugar())
14 print(p.jugar())
15 print(p.especie)
```

Salida real

```
Pyto juega. Energía: 3
Pyto juega. Energía: 1
desconocida
```

El ejemplo, línea por línea

1. especie es un atributo de clase, compartido.
2. `__init__` guarda nombre y energía en la instancia; energía tiene valor por defecto.
3. jugar modifica `self.energia` y devuelve un texto: cada llamada descuenta 2.

Consejos

- Nombres de clases en CamelCase (CuentaBancaria), métodos en snake_case.
- Si una clase solo tiene `__init__` y un método, quizá baste con una función.

Ejercicio

Modela una CuentaBancaria con titular y saldo, y métodos depositar, retirar y mostrar_saldo.

Errores frecuentes

Error frecuente	Cómo resolverlo
<code>TypeError: __init__() missing 1 required positional argument</code>	Creaste el objeto sin un dato obligatorio: <code>Tarea()</code> en lugar de <code>Tarea("título")</code> .
<code>NameError: name 'titulo' is not defined dentro de un método</code>	Dentro de los métodos, los atributos se leen con <code>self.titulo</code> .

Práctica evaluable

La clase Tarea • Pieza del proyecto: La clase Tarea

Pasa la tarea de diccionario a objeto. Crea la clase Tarea:

- `Tarea(titulo, prioridad="media")` guarda `titulo`, `prioridad` y `hecha = False`.
- `completar()` marca la tarea como hecha.
- `__str__` devuelve `"[ ] Comprar pan (alta)"`, o `"[x] Comprar pan (alta)"` si está hecha.
- `a_dict()` devuelve `{"titulo": ..., "prioridad": ..., "hecha": ...}`.

Plantilla para empezar

```

1 class Tarea:
2     def __init__(self, titulo, prioridad="media"):
3         self.titulo = titulo
4         # guarda la prioridad y hecha = False
5
6     def completar(self):
7         pass
8
9     def __str__(self):
10        return ""
11
12    def a_dict(self):
13        return {}

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Qué representa self?

- a) El módulo
- b) Una palabra reservada
- c) La instancia actual
- d) La clase

7

Programación orientada a objetos II

Herencia: una clase hija reutiliza y especializa a la madre. `super()` llama a la versión de la clase madre.

Polimorfismo: objetos distintos responden al mismo método a su manera. Puedes recorrerlos sin preguntar de qué tipo son.

Los métodos especiales (dunder) como `__str__`, `__repr__`, `__eq__` y `__len__` integran tus objetos con `print`, comparaciones y funciones de Python.

El concepto a fondo

La herencia permite crear una clase hija que reutiliza todo lo de la madre y agrega o cambia lo que necesita: `class TareaConFecha(Tarea):`. La hija tiene todos los métodos de la madre sin reescribirlos.

Sobrescribir es definir en la hija un método con el mismo nombre. Para extender en lugar de reemplazar se usa `super()`: `super().__init__(titulo)` ejecuta el inicializador de la madre y luego agregas lo nuevo.

El polimorfismo es tratar objetos de clases distintas de la misma forma: en el ejemplo, `a.hablar()` funciona igual para un perro o un gato, y cada uno responde a su manera.

`isinstance(obj, Clase)` pregunta si un objeto es de una clase o de alguna hija. Prefiere la composición (un objeto que contiene a otro) cuando la relación no es claramente "es un".

Ejemplo

```

1 class Animal:
2     def __init__(self, nombre):
3         self.nombre = nombre
4     def hablar(self):
5         return "..."
6     def __str__(self):
7         return f"{type(self).__name__}({self.nombre})"
8
9 class Perro(Animal):
10     def hablar(self):
11         return "¡Guau!"
12
13 class Gato(Animal):
14     def hablar(self):
15         return "Miau"
16
17 for a in [Perro("Toby"), Gato("Mia")]:
18     print(a, "->", a.hablar())

```

Salida real

```

Perro(Toby) -> ¡Guau!
Gato(Mia) -> Miau

```

El ejemplo, línea por línea

1. `Animal` define el comportamiento base y un `__str__` que usa el nombre de la clase real.
2. `Perro` y `Gato` sobrescriben `hablar`.
3. El `for` trata a ambos igual: polimorfismo.

Consejos

- Prefiere composición (un objeto contiene otro) sobre herencias profundas.
- `__repr__` es para desarrolladores; `__str__`, para usuarios.

Ejercicio

Crea `Empleado` y dos subclases (`Vendedor` con comisión, `Gerente` con bono) que calculen su sueldo de forma distinta.

Errores frecuentes

Error frecuente	Cómo resolverlo
<code>AttributeError: 'TareaConFecha' object has no attribute 'hecha'</code>	Te faltó llamar a <code>super().__init__(...)</code> en el <code>__init__</code> de la hija.
<code>RecursionError en __str__</code>	Dentro de <code>__str__</code> usaste <code>str(self)</code> : llama a <code>super().__str__()</code> en su lugar.

Práctica evaluable**Tareas con fecha límite** · Pieza del proyecto: **Herencia: TareaConFecha**

La clase Tarea ya existe (la de la lección anterior). Crea la clase hija TareaConFecha(Tarea):

- TareaConFecha(titulo, fecha_limite, prioridad="media"): llama a super().__init__ y guarda fecha_limite (un objeto date).
- vencida(hoy): True si la tarea no está hecha y hoy es posterior a la fecha límite.
- __str__: el texto de la madre más " · vence 01/10" (día/mes de la fecha límite).

Datos que prepara el evaluador (también prueba con otros valores)

```

1 class Tarea:
2     def __init__(self, titulo, prioridad="media"):
3         self.titulo = titulo
4         self.prioridad = prioridad
5         self.hecha = False
6
7     def completar(self):
8         self.hecha = True
9
10    def __str__(self):
11        marca = "x" if self.hecha else " "
12        return f"[{marca}] {self.titulo} ({self.prioridad})"
```

Plantilla para empezar

```

1 class TareaConFecha(Tarea):
2     def __init__(self, titulo, fecha_limite, prioridad="media"):
3         # llama al __init__ de la madre y guarda fecha_limite
4         pass
5
6     def vencida(self, hoy):
7         pass
8
9     def __str__(self):
10        return super().__str__()
```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Qué método especial usa print(obj)?

- __str__
- __print__
- __repr__ siempre
- __init__

8 Entornos virtuales y pip

Un entorno virtual es una instalación aislada de Python por proyecto. Evita que las librerías de un proyecto rompan otro.

pip instala paquetes desde PyPI. `requirements.txt` lista las dependencias para que cualquiera reproduzca tu entorno.

Herramientas modernas como `uv` o `poetry` hacen lo mismo más rápido y con archivos de bloqueo.

El concepto a fondo

Un entorno virtual (`python -m venv .venv`) es una carpeta con una instalación de Python propia del proyecto. Cada proyecto tiene sus librerías y versiones sin pisar a los demás.

Se activa con `.venv\Scripts\activate` en Windows o `source .venv/bin/activate` en macOS y Linux. Con el entorno activo, `pip install paquete` instala dentro del proyecto.

`pip freeze > requirements.txt` guarda la lista exacta de paquetes y versiones; `pip install -r requirements.txt` la reconstruye en otra computadora. Ese archivo es la receta de tu entorno.

El formato de `requirements.txt` es simple: una línea por paquete, `nombre==versión` para fijar una versión exacta, otros especificadores como `>=`, y comentarios con `#`. En esta práctica escribirás el lector de ese formato, la misma tarea que hace pip al leerlo.

Ejemplo (bash)

```
1 python -m venv .venv
2 # Windows: .venv\Scripts\activate
3 source .venv/bin/activate
4 pip install requests
5 pip freeze > requirements.txt
6 pip install -r requirements.txt
```

Resultado esperado

```
(.venv) $ <- el prompt muestra el entorno activo
Successfully installed requests-2.x ...
```

El ejemplo, línea por línea

1. `python -m venv .venv` crea el entorno.
2. Se activa según el sistema operativo.
3. `pip install` agrega una librería; `pip freeze` la anota con su versión exacta.
4. `pip install -r` instala todo lo anotado.

Consejos

- Agrega `.venv/` a tu `.gitignore`: el entorno no se sube al repositorio.
- Un entorno por proyecto, siempre.

Ejercicio

Crea un entorno para tu proyecto, instala 'rich' y úsalo para imprimir una tabla con colores.

Errores frecuentes

Error frecuente	Cómo resolverlo
pip instala pero Python no encuentra el módulo	Instalaste con el entorno desactivado o en otro Python. Activa el entorno y usa <code>python -m pip install</code> .
Subiste <code>.venv</code> al repositorio	Agrega <code>.venv/</code> a <code>.gitignore</code> : se comparte el <code>requirements.txt</code> , no el entorno.

Práctica evaluable

Leer un `requirements.txt` · Pieza del proyecto: **Lector de requirements**

Tu gestor documentará sus dependencias. Escribe:

- `parsear_requirements(texto)`: recibe el contenido de un `requirements.txt` y devuelve un diccionario `{paquete: versión}`. Los nombres van en minúsculas. Si la línea usa `==`, la versión es lo que sigue; si no tiene versión fija, el valor es `None`. Ignora líneas vacías, comentarios completos y comentarios al final de una línea.
- `linea_requirement(paquete, version=None)`: el camino inverso, `"requests==2.32.3"` o solo `"requests"`.

Plantilla para empezar

```

1 def parsear_requirements(texto):
2     """Contenido de requirements.txt -> {paquete: version o None}."""
3     dependencias = {}
4     for linea in texto.splitlines():
5         linea = linea.split("#")[0].strip() # quita comentarios y espacios
6         # salta las vacías; separa nombre y versión con "=="
7         return dependencias
8
9
10 def linea_requirement(paquete, version=None):
11     """{paquete, versión} -> línea de requirements.txt."""

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Para qué sirve `requirements.txt`?

- Configurar Git
- Documentar funciones
- Ejecutar tests
- Reproducir las dependencias del proyecto

9 Pruebas con assert y pytest

Un test es código que verifica que otro código funciona. `assert condición` lanza un error si la condición es falsa.

pytest descubre automáticamente archivos `test_*.py` y funciones `test_*`, y da reportes claros.

Escribir tests te da confianza para cambiar código sin miedo: si algo se rompe, te enteras al instante.

El concepto a fondo

Un test es código que verifica otro código. `assert condición` no hace nada si la condición es verdadera y lanza `AssertionError` si es falsa. Un buen test prueba un comportamiento concreto con un nombre que lo describe: `test_completar_marca_como_hecha`.

pytest busca archivos `test_*.py` y funciones `test_*` y las ejecuta todas con el comando `pytest`. Muestra qué tests pasaron y, de los que fallaron, qué valor se esperaba y cuál se obtuvo.

Un test vale por los errores que detecta. Por eso en esta práctica el evaluador corre tus tests dos veces: con la clase correcta (deben pasar) y con versiones que tienen errores escondidos (deben fallar). Si un test pasa con código roto, no está protegiendo nada.

Prueba los casos normales, los bordes (lista vacía, cero, el último elemento) y los valores por defecto. Escribir tests mientras programas te obliga a pensar qué debe hacer exactamente cada función.

Ejemplo

```
1 def es_par(n):
2     return n % 2 == 0
3
4 def test_es_par():
5     assert es_par(4)
6     assert not es_par(7)
7     assert es_par(0)
8
9     test_es_par()
10    print("Todos los tests pasaron")
```

Salida real

```
Todos los tests pasaron
```

El ejemplo, línea por línea

1. `es_par` es la función a probar.
2. `test_es_par` agrupa tres verificaciones, incluido el borde 0.
3. Llamamos al test a mano; con `pytest` se descubre y ejecuta solo.

Consejos

- Prueba los bordes: listas vacías, cero, negativos, textos con tildes.
- Ejecuta `pytest -q` antes de cada cambio importante.

Ejercicio

Escribe 5 tests para tu función `es_primo`, incluyendo 0, 1, 2 y un número grande.

Errores frecuentes

Error frecuente	Cómo resolverlo
Un test que nunca falla	Si el test pasa aunque rompas la función, no está verificando lo importante. Pruébalo rompiendo el código a propósito.
pytest no encuentra los tests	El archivo debe llamarse <code>test_algo.py</code> y las funciones empezar con <code>test_</code> .

Práctica evaluable

Escribe tus propios tests · Pieza del proyecto: **Tests de la clase Tarea**

La clase `Tarea` ya existe. Escribe al menos 4 funciones de test (con nombres que empiecen por `test_`) que usen `assert` para verificar:

1. que `completar()` marca la tarea como hecha;
2. el texto de una tarea pendiente: "[] Comprar pan (alta)";
3. el texto de una tarea hecha: "[x] ...";
4. que la prioridad por defecto es "media".

El evaluador correrá tus tests con la clase correcta (deben pasar) y con cuatro versiones con errores escondidos (cada una debe hacer fallar al menos un test).

Datos que prepara el evaluador (también prueba con otros valores)

```

1 class Tarea:
2     def __init__(self, titulo, prioridad="media"):
3         self.titulo = titulo
4         self.prioridad = prioridad
5         self.hecha = False
6
7     def completar(self):
8         self.hecha = True
9
10    def __str__(self):
11        marca = "x" if self.hecha else " "
12        return f"[{marca}] {self.titulo} ({self.prioridad})"
```

Plantilla para empezar

```

1 def test_completar_marca_como_hecha():
2     t = Tarea("Comprar pan", "alta")
3     t.completar()
4     assert t.hecha
5
6
7 # Escribe aquí los otros tres tests
```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Qué funciones descubre pytest como tests?

- a) Las que empiezan con test_
- b) Las que tienen assert
- c) Las decoradas con @test
- d) Todas

10 Depuración profesional

Depurar es encontrar por qué el código hace lo que hace. El método: reproducir el error, aislarlo, formular una hipótesis, comprobarla.

`breakpoint()` pausa la ejecución y abre el depurador `pdb`: con `n` avanzas, `s` entras a una función, `p` variable inspeccionas y `c` continuas.

Los editores (VS Code, PyCharm) ofrecen depuración visual con puntos de interrupción y observación de variables.

El concepto a fondo

Depurar es un método, no adivinar: 1) reproducir el error de forma confiable, 2) aislarlo en el caso más chico posible, 3) formular una hipótesis, 4) comprobarla mirando valores reales, y 5) corregir y agregar un test para que no vuelva.

La herramienta más simple es un `print` estratégico que muestre los valores intermedios. La más potente es `breakpoint()`: pausa el programa y abre el depurador (`pdb`), donde puedes inspeccionar variables (`p` variable), avanzar línea por línea (`n`) o continuar (`c`). Los IDE como VS Code ofrecen lo mismo con clics.

`assert` también sirve como red de seguridad dentro del código: documenta lo que debe ser cierto en ese punto y avisa apenas deja de serlo.

Los errores más traicioneros no lanzan excepciones: devuelven un resultado incorrecto. Un rango que empieza en 1 en lugar de 0, un `reverse=True` de más o una clave mal escrita. Por eso los tests con valores conocidos son tan valiosos.

Ejemplo

```

1 def promedio(valores):
2     assert valores, "La lista no puede estar vacía"
3     return sum(valores) / len(valores)
4
5 try:
6     promedio([])
7 except AssertionError as e:
8     print("Detectado:", e)
9
10 # breakpoint() # descomenta para pausar aquí y abrir pdb
11 print(promedio([4, 6, 8]))

```

Salida real

```

Detectado: La lista no puede estar vacía
6.0

```

El ejemplo, línea por línea

1. El `assert` documenta que la lista no puede estar vacía y lo verifica.
2. Se atrapa el `AssertionError` para mostrar el mensaje.
3. `breakpoint()` (comentado) pausaría ahí para inspeccionar.

Consejos

- Explicar el problema en voz alta (o a un patito de goma) resuelve más bugs de los que crees.
- Cambia una sola cosa a la vez y vuelve a probar.

Ejercicio

Toma un programa tuyo, introduce un bug a propósito y encuéntralo usando `breakpoint()`.

Errores frecuentes

Error frecuente	Cómo resolverlo
Corregir sin entender	Si cambias cosas al azar hasta que funcione, el error suele volver. Primero entiende por qué ocurre.
Errores de borde (off-by-one)	<code>range(1, len(lista))</code> saltea el primer elemento: revisa siempre dónde empiezan y terminan tus rangos.

Práctica evaluable

Depura el ordenador de tareas · Pieza del proyecto: **Orden y conteo depurados**

Este código tiene **tres errores**. No lanza excepciones: da resultados equivocados. Encuéntralos y corrígelos.

- `ordenar_por_prioridad(tareas)` debe ordenar alta → media → baja y, a igual prioridad, por título alfabético.
- `contar_pendientes(tareas)` debe contar las tareas no hechas.

Consejo: agrega un `print` dentro del bucle o de la `lambda` para ver qué valores se usan, y bórralo al terminar.

Plantilla para empezar

```

1  ORDEN = {"alta": 1, "media": 2, "baja": 2}
2
3
4  def ordenar_por_prioridad(tareas):
5      """alta -> media -> baja; a igual prioridad, por título."""
6      return sorted(tareas, key=lambda t: (ORDEN[t["prioridad"]], t["título"]),
7                    ↵ reverse=True)
8
9  def contar_pendientes(tareas):
10     """Cantidad de tareas no hechas."""
11     total = 0
12     for i in range(1, len(tareas)):
13         if not tareas[i]["hecha"]:
14             total += 1
15     return total

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

En un traceback, ¿dónde está el tipo de error?

- a) En el medio
- b) No aparece
- c) En la primera línea
- d) En la última línea

Proyecto integrador

Gestor de tareas de consola

Una aplicación CLI para crear, listar, completar y borrar tareas, con persistencia en JSON.

Paso a paso

1. Define una clase Tarea (título, prioridad, fecha, completada) y una clase GestorTareas.
2. Guarda y carga las tareas en un archivo JSON con pathlib.
3. Crea un menú en bucle con opciones numeradas y validación de entrada.
4. Maneja errores con excepciones propias (TareaNoEncontrada).
5. Escribe tests con pytest para las operaciones del gestor.
6. Extra: filtros por prioridad y exportación a CSV.

Criterios de éxito

Funciona sin errores con entradas normales y extrañas, está dividido en funciones o clases con nombres claros, tiene docstrings y tests donde corresponde, y registraste el proceso en la Plantilla de proyecto del Cuadernillo.

Checklist de cierre

- ☐ Puedo explicar y usar: comprensiones
- ☐ Puedo explicar y usar: funciones avanzadas
- ☐ Puedo explicar y usar: módulos y paquetes
- ☐ Puedo explicar y usar: archivos, csv y json
- ☐ Puedo explicar y usar: excepciones a fondo
- ☐ Puedo explicar y usar: programación orientada a objetos i
- ☐ Puedo explicar y usar: programación orientada a objetos ii
- ☐ Puedo explicar y usar: entornos virtuales y pip
- ☐ Puedo explicar y usar: pruebas con assert y pytest
- ☐ Puedo explicar y usar: depuración profesional
- ☐ Aprobé las 10 prácticas evaluables y mi proyecto ejecuta la demo
- ☐ Terminé el proyecto "Gestor de tareas de consola"

Tu proyecto del nivel, pieza a pieza

Gestor de tareas de consola

Una aplicación de consola para crear, listar, completar y guardar tareas en JSON. Cada lección del Nivel 2 aporta una pieza: comprensiones para filtrar, funciones flexibles, módulos, archivos, excepciones propias, clases, tests y depuración.

Cada práctica evaluable del nivel es una pieza de este programa. En la app, la sección **Mi proyecto** (desde el Árbol o el Inicio) une tus soluciones aprobadas en un solo archivo, `gestor_tareas.py`, te deja ejecutar la demo en el navegador cuando están todas y descargarlo en .zip (con un README) para abrirlo en tu IDE. Las piezas pendientes se reemplazan por versiones provisionales, así que puedes descargarlo en cualquier momento.

Las piezas

Lección	Pieza	Práctica
2.1 Comprensiones	Filtros con comprensiones	Filtrar tareas
2.2 Funciones avanzadas	Creación y orden de tareas	Crear tareas flexibles
2.3 Módulos y paquetes	Resumen y estadísticas	Resumen con la biblioteca estándar
2.4 Archivos, CSV y JSON	Persistencia en JSON	Guardar y cargar en JSON
2.5 Excepciones a fondo	Validación con excepciones	Validar tareas con excepciones propias
2.6 Programación orientada a objetos I	La clase Tarea	La clase Tarea
2.7 Programación orientada a objetos II	Herencia: TareaConFecha	Tareas con fecha límite
2.8 Entornos virtuales y pip	Lector de requirements	Leer un requirements.txt
2.9 Pruebas con assert y pytest	Tests de la clase Tarea	Escribe tus propios tests
2.10 Depuración profesional	Orden y conteo depurados	Depura el ordenador de tareas

El programa principal

Este es el código que Pythonia agrega al final del archivo: no tienes que escribirlo, pero vale la pena leerlo, porque muestra cómo se conectan tus piezas. `demo()` hace un recorrido automático y el bloque `if __name__ == "__main__":` es lo que corre al ejecutar el archivo en tu IDE.

```

1  # ===== Programa principal (escrito por Pythonia, usa tus piezas)
   ↪ =====
2  ARCHIVO = "tareas.json"
3
4
5  def cargar():
6      from pathlib import Path
7      ruta = Path(ARCHIVO)
8      return desde_json(ruta.read_text(encoding="utf-8")) if ruta.exists() else []
9
10
11 def guardar(tareas):
12     from pathlib import Path
13     Path(ARCHIVO).write_text(a_json(tareas), encoding="utf-8")
14
15
16 def mostrar(tareas):
17     for i, t in enumerate(ordenar_por_prioridad(tareas), start=1):
18         marca = "x" if t["hecha"] else " "
19         print(f"{i}. [{marca}] {t['titulo']} ({t['prioridad']})")
20
21
22 def demo():
23     """Recorrido automático por todas tus piezas."""
24     from datetime import date
25     tareas, errores = [], []
26     agregar_segura(tareas, crear_tarea("Comprar pan", "casa", prioridad="alta"),
   ↪ errores)
27     agregar_segura(tareas, crear_tarea("Pagar la luz", prioridad="alta"), errores)
28     agregar_segura(tareas, crear_tarea("Leer un capítulo", prioridad="baja"),
   ↪ errores)
29     agregar_segura(tareas, crear_tarea(" "), errores)
30     agregar_segura(tareas, crear_tarea("Llamar a Ana", prioridad="urgente"),
   ↪ errores)
31     tareas[1]["hecha"] = True
32     print("— Tareas ordenadas —")
33     mostrar(tareas)
34     print("Rechazadas:", errores)
35     print("Pendientes:", pendientes(tareas), "| contador:",
   ↪ contar_pendientes(tareas))

```

```

36     print("Resumen:", resumen(tareas), f"| {porcentaje_hecho(tareas)}% hecho")
37     print("JSON ida y vuelta sin pérdidas:", desde_json(a_json(tareas)) == tareas)
38     print("— Versión con clases —")
39     t = Tarea("Estudiar Python", "alta")
40     print(t)
41     t.completar()
42     print(t)
43     f = TareaConFecha("Entregar informe", date(2026, 10, 1))
44     print(f, "| vencida el 03/10:", f.vencida(date(2026, 10, 3)))
45     print("requirements:", parsear_requirements("requests==2.32.3\n#
↳ comentario\nRich"))
46     print("— Tus tests —")
47     for nombre, prueba in list(globals().items()):
48         if nombre.startswith("test_") and callable(prueba):
49             prueba()
50             print("✓", nombre)
51
52
53 def menu():
54     """Aplicación interactiva: ejecútala en tu IDE."""
55     tareas = cargar()
56     while True:
57         print("\n1) Ver  2) Agregar  3) Completar  4) Resumen  0) Salir")
58         opcion = input("Opción: ").strip()
59         if opcion == "1":
60             mostrar(tareas)
61         elif opcion == "2":
62             errores = []
63             nueva = crear_tarea(input("Título: "), prioridad=input("Prioridad
↳ (alta/media/baja): ").strip() or "media")
64             print("Agregada." if agregar_segura(tareas, nueva, errores) else f"No
↳ se pudo: {errores[-1]}")
65         elif opcion == "3":
66             ordenadas = ordenar_por_prioridad(tareas)
67             mostrar(tareas)
68             try:
69                 ordenadas[int(input("Número: ")) - 1]["hecha"] = True
70             except (ValueError, IndexError):
71                 print("Número inválido.")
72         elif opcion == "4":
73             print(resumen(tareas), f"{porcentaje_hecho(tareas)}% hecho")
74         elif opcion == "0":
75             guardar(tareas)
76             print("Guardado en", ARCHIVO)
77             break
78
79
80 if __name__ == "__main__":
81     menu()

```

Ejercicios con lente de especialización

Cada concepto del nivel aplicado a tu ruta. Elige la columna de tu ruta principal; si sigues el Modo Pythonista Total, resuelve las tres.

Automatización de oficina

Lección	Ejercicio con lente
1. Comprensiones	Con una comprensión, filtra de una lista de (archivo, mes) solo los .xlsx modificados este mes.
2. Funciones avanzadas	Escribe renombrar(*archivos, prefijo="", con_fecha=False) que devuelva los nombres nuevos sin tocar el disco.
3. Módulos y paquetes	Separa tu robot en lectura.py, proceso.py y reporte.py, con un main.py que los orqueste.
4. Archivos, CSV y JSON	Lee todos los CSV de una carpeta con pathlib y únelos en uno solo con un único encabezado.
5. Excepciones a fondo	Haz que tu robot salte los archivos dañados, los anote en una lista de errores y continúe con el resto.
6. Programación orientada a objetos I	Crea una clase Reporte con fecha y filas, y métodos agregar_fila y guardar_csv.
7. Programación orientada a objetos II	Crea Exportador con subclases ExportadorCSV y ExportadorExcel que compartan el método exportar().
8. Entornos virtuales y pip	Crea un entorno para tu robot con openpyxl y python-docx y genera su requirements.txt.
9. Pruebas con assert y pytest	Escribe tests para tu clasificador de archivos por extensión, incluyendo .PDF en mayúsculas y archivos sin extensión.
10. Depuración profesional	Pon un breakpoint() dentro del bucle que procesa planillas y encuentra en qué fila falla una conversión de número.

Videojuegos

Lección	Ejercicio con lente
1. Comprensiones	Genera con una comprensión 20 enemigos con posiciones aleatorias en la parte superior de la pantalla.
2. Funciones avanzadas	Escribe crear_enemigo(tipo, **atributos) que aplique valores por defecto y permita cambiar vida o velocidad.
3. Módulos y paquetes	Separa tu juego en config.py, entidades.py y main.py.
4. Archivos, CSV y JSON	Guarda y carga el récord del jugador en un archivo JSON.
5. Excepciones a fondo	Si falta una imagen, dibuja un rectángulo de color en su lugar en vez de cerrar el juego.
6. Programación orientada a objetos I	Crea la clase Jugador con vida, puntos y métodos recibir_daño y sumar_puntos.

Lección	Ejercicio con lente
7. Programación orientada a objetos II	Crea Enemigo y dos subclases (Zombi lento, Murciélago que zigzaguea) que sobrescriban mover().
8. Entornos virtuales y pip	Crea un entorno virtual para tu juego e instala pygame.
9. Pruebas con assert y pytest	Testea la lógica de colisiones y puntaje sin abrir ninguna ventana.
10. Depuración profesional	Depura por qué tu personaje atraviesa el suelo a alta velocidad imprimiendo su posición en cada cuadro.

Bases de datos

Lección	Ejercicio con lente
1. Comprensiones	Convierte las filas de una consulta (tuplas) en una lista de diccionarios con una comprensión.
2. Funciones avanzadas	Escribe consultar(tabla, **filtros) que arme el WHERE con parámetros ? de forma segura.
3. Módulos y paquetes	Separa conexión, esquema y consultas en db.py, esquema.py y consultas.py.
4. Archivos, CSV y JSON	Importa un CSV de productos a SQLite y exporta el resultado de una consulta a JSON.
5. Excepciones a fondo	Atrapa sqlite3.IntegrityError al insertar duplicados y muestra un mensaje claro.
6. Programación orientada a objetos I	Crea una clase BaseDeDatos que encapsule la conexión y ofrezca insertar y buscar.
7. Programación orientada a objetos II	Crea ConsultaBase y subclases para distintos reportes que compartan el método ejecutar().
8. Entornos virtuales y pip	Crea un entorno del proyecto con SQLAlchemy y DuckDB instalados.
9. Pruebas con assert y pytest	Escribe tests con una base :memory: nueva y limpia en cada prueba.
10. Depuración profesional	Activa con set_trace_callback el registro de cada SQL que ejecuta sqlite3.

Solucionario de autoevaluación

- 1. Comprensiones:** d) ['h','l','a']. Recorre cada carácter y descarta la 'o'. El resultado es una lista.
- 2. Funciones avanzadas:** c) 12. Se define y se llama en el acto con x=3, y=4.
- 3. Módulos y paquetes:** d) "__main__". Cuando se importa, vale el nombre del módulo; cuando se ejecuta, "__main__".
- 4. Archivos, CSV y JSON:** a) Cierra el archivo automáticamente, incluso con errores. El context manager garantiza el cierre del archivo pase lo que pase.
- 5. Excepciones a fondo:** d) Solo si no hubo excepción. else es el camino feliz; finally es el que corre siempre.
- 6. Programación orientada a objetos I:** c) La instancia actual. self es el objeto concreto sobre el que se llama el método.
- 7. Programación orientada a objetos II:** a) __str__. print usa __str__; si no existe, recurre a __repr__.
- 8. Entornos virtuales y pip:** d) Reproducir las dependencias del proyecto. Con pip install -r requirements.txt otra persona obtiene las mismas librerías.
- 9. Pruebas con assert y pytest:** a) Las que empiezan con test_. Convención: archivos test_*.py y funciones test_*.
- 10. Depuración profesional:** d) En la última línea. La última línea muestra el tipo y el mensaje; lo de arriba es el recorrido.

Pistas y soluciones de las prácticas

Usa las pistas de a una, y solo después de intentarlo. Hay más de una solución correcta: si la tuya pasa las pruebas, compárala con esta para aprender otra forma de pensarlo.

1. Comprensiones · Filtrar tareas

Pista 1: Lista: [...], diccionario: {clave: valor ...}, conjunto: {valor ...}.

Pista 2: Para filtrar, el if va al final de la comprensión.

```

1 def pendientes(tareas):
2     """Títulos de las tareas que no están hechas."""
3     return [t["titulo"] for t in tareas if not t["hecha"]]
4
5
6 def por_prioridad(tareas, prioridad):
7     """Tareas (diccionarios completos) con esa prioridad."""
8     return [t for t in tareas if t["prioridad"] == prioridad]
9
10
11 def indice(tareas):
12     """Diccionario {titulo: tarea}."""
13     return {t["titulo"]: t for t in tareas}
14
15
16 def prioridades_usadas(tareas):
17     """Conjunto de prioridades presentes."""
18     return {t["prioridad"] for t in tareas}

```

Por qué funciona: Cuatro comprensiones, cuatro estructuras. Ninguna modifica la lista original: crean colecciones nuevas, lo que hace estas funciones seguras de usar en cualquier parte del programa.

2. Funciones avanzadas · Crear tareas flexibles

Pista 1: etiquetas llega como tupla: conviértela con `list(etiquetas)`.

Pista 2: `update` agrega al diccionario todas las claves de `extra`.

```

1 def crear_tarea(titulo, *etiquetas, prioridad="media", **extra):
2     """Crea el diccionario de una tarea."""
3     tarea = {"titulo": titulo.strip(), "prioridad": prioridad, "hecha": False}
4     tarea["etiquetas"] = list(etiquetas)
5     tarea.update(extra)
6     return tarea
7
8
9 def ordenar(tareas, por="titulo", descendente=False):
10     """Lista nueva ordenada por la clave `por`."""
11     return sorted(tareas, key=lambda t: t[por], reverse=descendente)

```

Por qué funciona: La firma con `*etiquetas` y `**extra` hace la función flexible sin perder claridad: lo esencial tiene nombre propio y lo opcional se acomoda solo. `sorted` con `key` y `reverse` ordena por cualquier campo en una línea.

3. Módulos y paquetes · Resumen con la biblioteca estándar

Pista 1: `Counter(t["prioridad"] for t in tareas)` cuenta en una línea.

Pista 2: Una lista vacía es falsa: `if not tareas: return 0`.

```

1  from collections import Counter
2  import math
3
4
5  def resumen(tareas):
6      """Totales y conteo por prioridad."""
7      hechas = sum(1 for t in tareas if t["hecha"])
8      return {
9          "total": len(tareas),
10         "hechas": hechas,
11         "pendientes": len(tareas) - hechas,
12         "por_prioridad": dict(Counter(t["prioridad"] for t in tareas)),
13     }
14
15
16 def porcentaje_hecho(tareas):
17     """Porcentaje entero (hacia abajo); 0 si no hay tareas."""
18     if not tareas:
19         return 0
20     hechas = sum(1 for t in tareas if t["hecha"])
21     return math.floor(hechas * 100 / len(tareas))
22
23
24 def fecha_corta(fecha):
25     """date -> 'dd/mm/aaaa'."""
26     return fecha.strftime("%d/%m/%Y")

```

Por qué funciona: Tres módulos estándar resuelven en pocas líneas lo que a mano llevaría bucles y contadores: `Counter` cuenta, `math.floor` redondea hacia abajo y `strftime` da formato a fechas.

4. Archivos, CSV y JSON · Guardar y cargar en JSON

Pista 1: `dumps` y `loads` trabajan con texto; `dump` y `load`, con archivos abiertos.

Pista 2: `isinstance(datos, list)` comprueba el tipo.

```

1  import json
2
3
4  def a_json(tareas):
5      """Lista de tareas -> texto JSON legible."""
6      return json.dumps(tareas, ensure_ascii=False, indent=2)
7
8
9  def desde_json(texto):
10     """Texto JSON -> lista de tareas ([] si está vacío)."""
11     if not texto.strip():
12         return []
13     datos = json.loads(texto)
14     if not isinstance(datos, list):
15         raise ValueError("El archivo no contiene una lista de tareas")
16     return datos

```

Por qué funciona: Separar la conversión (texto ↔ datos) de la lectura del disco hace estas funciones fáciles de probar sin archivos reales. El programa principal se encarga de `Path.read_text` y `write_text`.

5. Excepciones a fondo · Validar tareas con excepciones propias

Pista 1: `if tarea["prioridad"] not in PRIORIDADES:`

Pista 2: `str(e)` da el mensaje con el que se lanzó la excepción.

```

1  PRIORIDADES = ("alta", "media", "baja")
2
3
4  class TareaInvalida(Exception):
5      """Error propio del gestor de tareas."""
6
7
8  def validar(tarea):
9      if not tarea["titulo"].strip():
10         raise TareaInvalida("La tarea necesita un título")
11      if tarea["prioridad"] not in PRIORIDADES:
12         raise TareaInvalida(f"Prioridad inválida: {tarea['prioridad']}")
13      return True
14
15
16 def agregar_segura(tareas, tarea, errores):
17     """Agrega si es válida; si no, anota el error. Devuelve True o False."""
18     try:
19         validar(tarea)
20     except TareaInvalida as e:
21         errores.append(str(e))
22         return False
23     else:
24         tareas.append(tarea)
25         return True

```

Por qué funciona: `validar` solo detecta y lanza; `agregar_segura` decide qué hacer con el problema. Esa separación es la esencia del manejo de errores: quien detecta no siempre es quien sabe cómo reaccionar. El `else` deja claro que `agregar` solo ocurre si la validación pasó.

6. Programación orientada a objetos I · La clase Tarea

Pista 1: Todo lo que la tarea "recuerda" va en `self`.

Pista 2: `marca = "x" if self.hecha else " "`

```

1 class Tarea:
2     def __init__(self, titulo, prioridad="media"):
3         self.titulo = titulo
4         self.prioridad = prioridad
5         self.hecha = False
6
7     def completar(self):
8         self.hecha = True
9
10    def __str__(self):
11        marca = "x" if self.hecha else " "
12        return f"[{marca}] {self.titulo} ({self.prioridad})"
13
14    def a_dict(self):
15        return {"titulo": self.titulo, "prioridad": self.prioridad, "hecha":
            ↪ self.hecha}

```

Por qué funciona: La clase junta los datos y las acciones que los modifican. Quien usa una Tarea llama a `completar()` sin saber cómo se guarda el estado por dentro: eso es encapsular.

7. Programación orientada a objetos II · Tareas con fecha límite

Pista 1: Sin `super().__init__`, la hija no tendría `hecha` ni `titulo`.

Pista 2: Las fechas se comparan con `>` igual que los números.

```

1 class TareaConFecha(Tarea):
2     def __init__(self, titulo, fecha_limite, prioridad="media"):
3         super().__init__(titulo, prioridad)
4         self.fecha_limite = fecha_limite
5
6     def vencida(self, hoy):
7         return not self.hecha and hoy > self.fecha_limite
8
9     def __str__(self):
10        return super().__str__() + f" · vence {self.fecha_limite.strftime('%d/%m')}"

```

Por qué funciona: La hija solo escribe lo nuevo: la fecha, la regla de vencimiento y el agregado al texto. Todo lo demás (`completar`, la marca `[x]`) lo hereda. Si mañana cambias el formato en `Tarea`, `TareaConFecha` se actualiza sola.

8. Entornos virtuales y pip · Leer un `requirements.txt`

Pista 1: `linea.split("==", 1)` separa en dos partes como máximo.

Pista 2: Para `>=`, quédate con lo que está antes del primer símbolo `<`, `>`, `=`, `!` o `~`.

```

1  import re
2
3
4  def parsear_requirements(texto):
5      """Contenido de requirements.txt -> {paquete: version o None}."""
6      dependencias = {}
7      for linea in texto.splitlines():
8          linea = linea.split("#")[0].strip()
9          if not linea:
10             continue
11          if "==" in linea:
12              nombre, version = linea.split("==", 1)
13              dependencias[nombre.strip().lower()] = version.strip()
14          else:
15              nombre = re.split(r"[<=&;!~]", linea)[0]
16              dependencias[nombre.strip().lower()] = None
17      return dependencias
18
19
20 def linea_requirement(paquete, version=None):
21     """{paquete, versión} -> línea de requirements.txt."""
22     return f"{paquete}=={version}" if version else paquete

```

Por qué funciona: Leer un formato de texto línea por línea es una de las tareas más frecuentes: limpiar, saltar lo que no interesa y separar las partes. `continue` evita anidar el resto del bucle dentro de un `if`.

9. Pruebas con `assert` y `pytest` · Escribe tus propios tests

Pista 1: Compara el texto completo con `==`, no solo una parte.

Pista 2: Para el test 3, completa la tarea antes de mirar su texto.

```

1  def test_completar_marca_como_hecha():
2      t = Tarea("Comprar pan", "alta")
3      t.completar()
4      assert t.hecha
5
6
7  def test_texto_pendiente():
8      assert str(Tarea("Comprar pan", "alta")) == "[ ] Comprar pan (alta)"
9
10
11 def test_texto_hecha():
12     t = Tarea("Comprar pan", "alta")
13     t.completar()
14     assert str(t) == "[x] Comprar pan (alta)"
15
16
17 def test_prioridad_por_defecto():
18     assert Tarea("Leer").prioridad == "media"

```

Por qué funciona: Cada test verifica una sola cosa y su nombre la describe. Comparar el texto exacto (y no solo que contenga algo) es lo que permite atrapar errores sutiles, como una prioridad que desaparece.

10. Depuración profesional · Depura el ordenador de tareas

Pista 1: Errores: uno en el diccionario, uno en sorted y uno en el range.

Pista 2: Recorrer directamente `for tarea in tareas` evita los errores de índices.

```
1 ORDEN = {"alta": 1, "media": 2, "baja": 3}
2
3
4 def ordenar_por_prioridad(tareas):
5     """alta -> media -> baja; a igual prioridad, por título."""
6     return sorted(tareas, key=lambda t: (ORDEN[t["prioridad"]], t["titulo"]))
7
8
9 def contar_pendientes(tareas):
10    """Cantidad de tareas no hechas."""
11    total = 0
12    for tarea in tareas:
13        if not tarea["hecha"]:
14            total += 1
15    return total
```

Por qué funciona: Los tres errores eran silenciosos: baja compartía número con media, `reverse=True` invertía todo el orden y `range(1, ...)` saltaba la primera tarea. Recorrer los elementos en lugar de los índices elimina de raíz el tercero.